Mathematical Modeling of Team Success in Baseball: A Sabermetric Approach

Kyla Igawa, Applied Mathematics B.S.
Wentworth Institute of Technology

Revisiting the Pythagorean Won-Loss Formula

One of the most enduring formulas in sports analytics is the Pythagorean Won-Loss Formula, which estimates a teams winning percentage as:

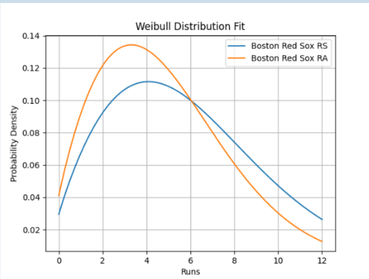
$$\boldsymbol{Win}\% = \frac{RS_{obs}^{\gamma}}{RS_{obs}^{\gamma} + RA_{obs}^{\gamma}}$$

Originally introduced by Bill James with $\gamma = 0$, the Pythagorean formula links offensive and defensive output to a team's win percentage. Later analysis showed that the exponent γ can be optimized, and that the formula has a probabilistic foundation.

By modeling runs scored and allowed as independent Weibull-distributed variables, the win probability naturally arises from the chance that one exceeds the other. The Weibull's flexibility makes it well-suited to the skewed shape of scoring data.

The form of the Weibull distribution used is:

$$f(x;\ \alpha,\beta,\gamma) = \begin{cases} \frac{\gamma}{\alpha}\left(\frac{x-\beta}{\alpha}\right)^{\gamma-1} e^{-\left(\frac{x-\beta}{\alpha}\right)^{\gamma}} & if\ x \geq \beta \\ 0 & otherwise \end{cases}$$



By treating runs scored and allowed as independent Weibull random variables, we can derive a new version of the formula – grounded in probability theory rather than intuition. Here's how it arises mathematically.

$$Prob(X > Y) = \int_{x=\beta}^{\infty}\int_{y=\beta}^{x} f(x{:}\,\alpha_{RS},\beta,\gamma) f(y{:}\,\alpha_{RA},\beta,\gamma)\,dy\,dx$$

$$\int_{x=\beta}^{\infty}\int_{y=\beta}^{x} \frac{\gamma}{\alpha_{RS}}\left(\frac{x-\beta}{\alpha_{RS}}\right)^{\gamma-1} e^{-\left(\frac{x-\beta}{\alpha_{RS}}\right)^{\gamma}} \times \frac{\gamma}{\alpha_{RA}}\left(\frac{y-\beta}{\alpha_{RA}}\right)^{\gamma-1} e^{-\left(\frac{y-\beta}{\alpha_{RA}}\right)^{\gamma}}\,dy\,dx$$

$$\int_{x=0}^{\infty} \frac{\gamma}{\alpha_{RS}}\left(\frac{x-\beta}{\alpha_{RS}}\right)^{\gamma-1} e^{-\left(\frac{x-\beta}{\alpha_{RS}}\right)^{\gamma}} \left[\int_{y=0}^{x} \frac{\gamma}{\alpha_{RA}}\left(\frac{y-\beta}{\alpha_{RA}}\right)^{\gamma-1} e^{-\left(\frac{y-\beta}{\alpha_{RA}}\right)^{\gamma}} dy\right] dx$$

Through a sequence of algebraic steps and calculus-based simplifications, this expression reduces neatly to the following compact formula for winning percentage:

$$\boldsymbol{Win}\ \% = \frac{(RS-\beta)^{\gamma}}{(RS-\beta)^{\gamma} + (RA-\beta)^{\gamma}}$$

Abstract

This project explored and extended the Pythagorean Win-Loss formula by modeling baseball team performance using season and game-level data. Building on a Weibull-based model optimized via least squares, the original methodology was replicated and further refined. The focus then shifted to simulating individual games through regression techniques, incorporating offensive and pitching stats. Feature engineering and model comparison led to a flexible framework for predicting outcomes, bridging theory with applied machine learning in sports analytics.

Modeling Team Performance using The Method of Least Squares

We applied the Weibull-based Pythagorean won-loss formula to all 14 teams in the 2004 American League season by modeling run distributions using Weibull functions. Optimal parameters were estimated via least squares by minimizing the squared error between observed and expected run frequencies across binned data.

$$\boldsymbol{SSE} = \sum_{i} (O_i - E_i)^2$$

$$\boldsymbol{SSE\ (RS)} = \sum_{k=1}^{\#\,Bins} \left(RS_{obs}(k) - \#Games \cdot A(\alpha_{RS}, -.5, \gamma, k)\right)^2$$

We use Broyden-Fletcher-Goldfarb-Shanno algorithm, a quasi-Newton method, to minimize the SSE. BFGS iteratively updates estimates of the best-fit parameters by using gradient and curvature (Hessian) information.

Compute f(x_k) and the gradient:

$$g_k = \Delta f(x_k)$$

→ Compute the search direction (the direction to move):

$$d = -H^{-1}\Delta f(x_k)$$

Compute:

$$s_k = x_{k+1} - x_k$$
$$y_k = \Delta f(x_{k+1}) - \Delta f(x_k)$$

→ Plug in to updated Hessian Approximation

$$H_{k+1} = H_k - \frac{H_k s_k s_k^T H_k}{s_k^T H_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}$$

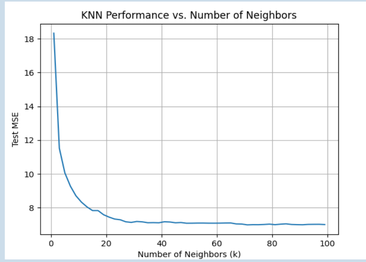
Repeat until convergence criteria is met:
1. Step size is tiny
2. Gradient norm is tiny
3. The change in function value is negligible

Model Testing and Selection

To evaluate different modeling approaches for predicting runs scored in simulated baseball games, we split the data into 80% training and 20% testing sets. Several regression techniques were explored, including K-Nearest-Neighbors (KNN), linear regression, and polynomial regression. We tested their performance by calculating their mean squared error (MSE).

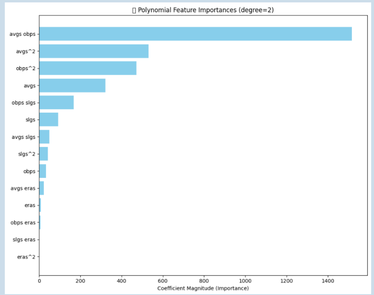
K-Nearest-Neighbors (KNN):

KNN is a non-parametric regression method that predicts a value based on the average of its closest training examples. The graph illustrates how model performance varied as we tuned the number of neighbors (k), with an optimal value minimizing the test error while avoiding overfitting.



Polynomial Regression:

Polynomial regression extends linear models by adding higher-degree and interaction terms. This feature expansion helped capture nonlinear relationships between batting and pitching stats.



Mean Squared Error Results:

Simulating Individual Games: A Regression Approach

To move beyond season-level predictions, I developed a simulation-based method to forecast individual baseball games -- valuable for sports analytics and betting. A custom 50-line simulator modeled games inning-by-inning using offensive stats (batting average, on base percentage, and slugging percentage) as well as the opposing team's ERA to probabilistically generate outcomes and estimate runs.

To model the relationship between team stats and simulated scoring, we applied linear regression on a polynomial expanded feature set – capturing nonlinear interactions (e.g., slg*era) while preserving interpretability. This framework showed strong performance and is adaptable to include additional variables such as bullpen strength, home-field advantage, weather conditions, etc.

```
Run    Linear Regression
C:\Users\15623\PyCharmMiscProject\.venv\Scripts\python.exe "C:\Users\
Enter Team 1 name: Los Angeles Dodgers
Enter Team 2 name: Boston Red Sox

Enter the following team stats for the Los Angeles Dodgers:
  Team Batting Average (AVG): .255
  Team On-Base Percentage (OBP): .330
  Team Slugging Percentage (SLG): .442
  Opposing Starting Pitcher's ERA: 2.23

Enter the following team stats for the Boston Red Sox:
  Team Batting Average (AVG): .253
  Team On-Base Percentage (OBP): .323
  Team Slugging Percentage (SLG): .432
  Opposing Starting Pitcher's ERA: 2.63

Predicted Runs Scored:
Los Angeles Dodgers: 5.41
Boston Red Sox: 4.99

Win Probabilities:
Los Angeles Dodgers: 60.34%
Boston Red Sox: 39.66%

Predicted Winner: Los Angeles Dodgers
```

Starting Pitchers:
Red Sox: Garrett Crochet
Los Angeles Dodgers: Yoshinobu Yamamoto

References

[Miller] A Derivation of the Pythagorean Won-Loss Formula in Baseball
https://web.williams.edu/Mathematics/sjmiller/public_html/399/handouts/PythagWonLoss_Paper.pdf
[Honorio] The BFGS Method
https://www.cs.purdue.edu/homes/jhonorio/16spring-cs52000-quasinewton.pdf
[Wikipedia] Least Squares
https://en.wikipedia.org/wiki/Least_squares
[Wikipedia] Probability Density Function
https://en.wikipedia.org/wiki/Probability_density_function
[Baseball Almanac] The History of Major League Baseball
https://www.baseball-almanac.com/